

SECUREOCTE

Real-Time Safety & Alert Processing Platform

Technical Architecture & Software Documentation

Document Classification: CONFIDENTIAL

Version: 3.0 (Production-Hardened)

Date: June 2025

Author: Manne Nikhil — Backend & Systems Engineer

1. Executive Summary

SecureOcte is a production-grade, real-time personal safety platform engineered with a hybrid edge + cloud architecture. The system is purpose-built to detect, process, and respond to user safety events with sub-second latency and enterprise-level reliability.

Primary Use Case: Women's personal safety — enabling SOS alerts, cab journey monitoring, walk monitoring, patrol dispatch coordination, and AI-assisted companionship.

1.1 Platform Scope

Component	Technology	Role
Mobile App (Edge)	React Native (Expo 54)	Sensor fusion, SOS trigger, GPS tracking, user interface
Backend API	Node.js + Express	Alert ingestion, queue processing, auth, dispatch
Queue System	BullMQ + Redis	Decoupled async alert processing with retry
Database	MongoDB Atlas	Persistent storage for all entities
AI Layer	Google Gemini 2.5 Flash	Chatbot (Neha), voice command parsing, mode detection
Dashboard	Static HTML + JS	Operator control plane for real-time monitoring

1.2 Key Engineering Highlights

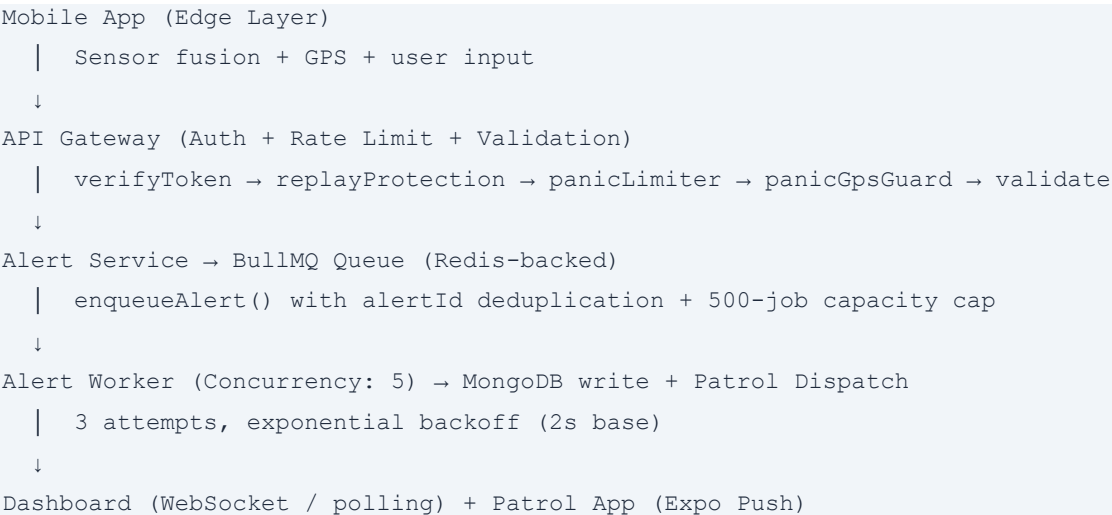
- Hybrid edge + cloud: latency-critical detection runs on-device; persistence and dispatch run server-side
- Queue-backed panic processing with job deduplication (alertId as BullMQ jobId)
- Multi-layer security: JWT + device binding + replay protection + GPS anomaly guards
- Fail-closed Redis dependency: rate limiting and replay protection block requests when Redis is unavailable
- PM2 cluster mode with per-core workers; Redis-backed shared rate-limit counters
- TTL-indexed MongoDB collections: panic and incident records auto-expire after 7 days; live streams after 24 hours

2. System Architecture

2.1 Architecture Overview

SecureOcte follows a layered, event-driven architecture with clear separation between the edge (mobile), processing (backend), and control (dashboard) layers.

Data Flow Pipeline:



2.2 Deployment Architecture

The backend runs under PM2 in cluster mode, spawning one Node.js process per CPU core. All instances share the same Redis instance for distributed rate limiting, replay protection, and GPS caching, ensuring consistent enforcement regardless of which worker handles a given request.

Layer	Technology	Scale Mode	Notes
Backend API	Node.js + Express	PM2 cluster (max cores)	512 MB memory restart guard
Queue Workers	BullMQ Worker	Concurrency = 5 per process	Processes panic-alerts queue
Rate Limiting	Redis + rate-limit-redis	Shared across all PM2 workers	Fail-closed on Redis unavailability
Database	MongoDB Atlas	Managed Atlas cluster	TTL indexes on time-sensitive collections
Cache/Queue store	Redis (ioredis)	Single-node or cluster mode	Exponential reconnect, max 10 retries

2.3 Environment Variables

The following environment variables must be configured in the backend .env file:

Variable	Required	Description
MONGO_URI	Yes	MongoDB Atlas connection string
JWT_SECRET	Yes	HMAC-SHA256 signing key for JWT tokens
REDIS_URL	Yes	Redis connection URL (fail-closed if absent)
GEMINI_API_KEY	Yes	Google Gemini AI API key
GOOGLE_MAPS_KEY	Yes	Google Maps API key for geocoding
GOOGLE_SPEECH_API_KEY	Yes	Google Speech-to-Text API key
ADMIN_USERNAME	Yes	Admin dashboard login credential
ADMIN_PASSWORD	Yes	Admin dashboard login credential
PUBLIC_BASE_URL	Yes	Public base URL for stream link generation
CORS_ORIGIN	Yes	Primary allowed CORS origin
ALLOWED_HOST	No	Host header whitelist (default: securebackend.in)
REDIS_CLUSTER	No	Set to 'true' to enable Redis cluster mode
REDIS_CLUSTER_NODES	No	Comma-separated host:port pairs for cluster
REDIS_TLS	No	Set to 'true' to enable TLS for Redis
HEALTH_TOKEN	No	Bearer token to access full /health response
EXTRA_CORS_ORIGINS	No	Comma-separated additional CORS origins

3. Backend Service

3.1 Technology Stack

Package	Version	Purpose
express	^4.18.2	HTTP server and routing framework
mongoose	^7.6.0	MongoDB ODM
bullmq	^5.7.0	Redis-backed job queue with retry and deduplication
ioredis	^5.3.2	Redis client (single-node + cluster support)
jsonwebtoken	^9.0.2	JWT signing and verification
bcryptjs	^3.0.3	Password hashing (bcrypt, 12 rounds)
helmet	^8.0.0	HTTP security headers
express-rate-limit	^7.5.0	In-process rate limiting (Redis-backed via rate-limit-redis)
rate-limit-redis	^4.2.0	RedisStore adapter for express-rate-limit
zod	^3.23.8	Schema validation for request payloads
pino	^8.21.0	Structured JSON logging
uuid	^11.1.0	UUID v4 generation for alertId and streamId
@google/genai	^1.34.0	Google Gemini AI SDK
socket.io	^4.7.5	WebSocket server for real-time events
multer	^2.0.2	Multipart file handling
ffmpeg-static + fluent-ffmpeg	^5.3.0 / ^2.1.3	Audio/video stream processing
morgan	^1.10.0	HTTP request logging
cors	^2.8.5	CORS middleware
dotenv	^16.3.1	Environment variable loading

3.2 Server Bootstrap Sequence

1. dotenv.config() loads environment variables
2. Express app created with trust proxy = 1 (required behind reverse proxy)
3. Host validation middleware: rejects requests not matching ALLOWED_HOST (except /stream/*)
4. CORS configured: wildcard for /stream, relaxed for /api/stream, strict for all other routes
5. Helmet applied: strict CSP for protected routes, permissive CSP for stream viewer
6. Body parser: 100 KB JSON limit
7. 10-second request timeout enforced globally

8. Distributed Redis rate limiting applied globally (120 req/min per IP)
9. Structured request logging (pino) with method, path, status, latency, IP
10. Static files served from /public
11. All route handlers mounted under /api/*
12. Global error handler registered last
13. MongoDB connection established; on success: ensureIndexes() + startAlertWorker()
14. Server starts listening on PORT (default 4000)

3.3 API Route Map

Base Path	File	Access Control	Purpose
/api/auth	routes/auth.js	Public + JWT	User signup, login, admin login, profile management
/api/panic	routes/panicRoutes.js	JWT + Device binding	SOS trigger, live location update, alert listing
/api/walk	routes/walk.js	JWT	Walk monitoring session CRUD
/api/cab-escort	routes/cabEscortRoutes.js	JWT	Cab trip monitoring, police support, location updates
/api/patrol	routes/Patrolroutes.js	JWT	Patrol vehicle trips, dispatch, acknowledgement
/api/incident	routes/incidentRoutes.js	JWT	Citizen incident reporting, nearest patrol dispatch
/api/stream	routes/stream.js	JWT (create) / Public (get)	Live stream creation and location polling
/api/stats	routes/stats.js	Admin only	Dashboard statistics (today, week, month, lifetime)
/api/speech	routes/speech.js	JWT	Speech processing routes
/api/voice	routes/voice.js	JWT	Voice command routes
/api/secureme	routes/secureme.cjs	JWT	SecureMe feature (biometric safety check)
/ai	routes/ai.js	JWT	Gemini AI chatbot (Neha) and voice parsing
/health	monitoring/healthRoute.js	Localhost / HEALTH_TOKEN	DB + Redis + Queue liveness probe
/stream/:streamId	Static HTML	Public	Stream viewer HTML page

4. Data Models

4.1 User

Collection: users | Auth schema for registered app users with embedded refresh token sub-documents.

Field	Type	Constraints	Notes
username	String	Required, 3–50 chars	Display name
password	String	Required	bcrypt hash, 12 rounds
mobile	String	Required, unique, 10 digits	Primary login identifier
trustedContacts	[String]	Optional	Array of contact phone numbers
gender	String	Enum: male/female/other	Optional profile field
dob	Date	Optional, age 10–120	Used to compute virtual 'age' field
refreshTokens	[RefreshToken]	Sub-document array	Token hash + deviceId + expiry + revoked flag

4.2 Panic (Alert)

Collection: panics | Created by the alert worker on SOS trigger. TTL: 7 days.

Field	Type	Notes
userId	String	References User._id
username	String	Denormalized for dashboard display
location.lat	Number	Latitude at time of panic
location.lng	Number	Longitude at time of panic
createdAt	Date	TTL index — auto-deleted after 7 days (604,800 seconds)

4.3 UserLocation

Collection: userlocations | Live tracking document — overwritten on each PUT /api/panic/location. Persists the latest known position for dashboard map display.

Field	Type	Notes
userId	String	References User._id
username	String	Populated at panic trigger time; back-filled on read if null
latitude / longitude	Number	Current position
alertId	String	UUID v4; ties location to a panic event

Field	Type	Notes
alertReason	String	Enum: MANUAL_PANIC / SAFETY_CHECK_NO_RESPONSE / SAFETY_CHECK_NO / OTHER

Indexes: { userId, updatedAt } compound, { alertId } single.

4.4 WalkSession

Collection: walksessions | Tracks each walk monitoring session.

Field	Type	Notes
userId / username	String	Owner identity
startLocation	{ latitude, longitude }	Coordinates at session start
status	String	Enum: ACTIVE / STOPPED
startedAt / stoppedAt	Date	Session timestamps

4.5 CabEscortTrip

Collection: cabescorttrips | Tracks active cab journeys. A new document is created on each /start and each police support trigger.

Field	Type	Notes
userId / username	String	Trip owner
destination	{ latitude, longitude, address }	Target drop-off from Google Places
vehicleNumber / vehicleType	String	Vehicle identification
currentLocation	{ latitude, longitude }	Overwritten every ~5 seconds
policeSupportRequested	Boolean	True when emergency alert created
status	String	Enum: active / completed

Index: { userId, status } compound. TTL: none — retained for audit.

4.6 PatrolTrip

Collection: patroltrips | Tracks patrol officer field activity and dispatched alerts.

Field	Type	Notes
officerId / officerName / badgeNumber	String	Officer identity

Field	Type	Notes
vehicleNumber / vehicleType / patrolZone	String	Vehicle and zone metadata
currentLocation	{ latitude, longitude }	Overwritten every 5 seconds via /update-location
expoPushToken	String	Expo push token for dispatch notifications
dispatchedAlert	Embedded Object	Fields: latitude, longitude, userName, alertType, dispatchedAt, acknowledged
status	String	Enum: active / completed

4.7 IncidentReport

Collection: incidentreports | Citizen-reported public safety incidents. TTL: 7 days.

Field	Type	Notes
category	String	Enum: Public Disturbance / Suspicious Gathering / Risky Behavior
crowdSize	String	Enum: Single person / Small group (2–5) / Medium group (6–10) / Large group (10+)
behaviorIndicators	[String]	Category-specific validated indicator strings
note	String	Free-text, max 500 chars
location	{ latitude, longitude }	Incident GPS coordinates
assignedPatrolTripId	String	Trip ID of nearest active patrol vehicle
assignedOfficerName	String	Denormalized officer name
status	String	Enum: pending / acknowledged / resolved

4.8 LiveStream

Collection: livestreams | Maps a streamId to a userId for the stream viewer. TTL: 24 hours.

Field	Type	Notes
streamId	String	UUID v4, unique index
userId	String	References User._id
createdAt	Date	Stream creation timestamp
expiresAt	Date	Default: createdAt + 24 hours; TTL index drives auto-deletion

4.9 Database Index Summary

Collection	Index	Type	Purpose
panics	{ userId, createdAt: -1 }	Compound	Fast user alert lookup
panics	{ createdAt: 1 }	TTL (7d)	Auto-expiry
userlocations	{ userId, updatedAt: -1 }	Compound	Latest location fetch
userlocations	{ alertId: 1 }	Single	Alert-based lookup
livestreams	{ streamId: 1 }	Unique	Stream resolution
livestreams	{ expiresAt: 1 }	TTL (field-driven)	24-hour auto-expiry
patroltrips	{ userId, isActive: 1 }	Compound	Active patrol filter
cabescorttrips	{ userId, status: 1 }	Compound	User trip history
incidentreports	{ userId: 1 }	Single	User report lookup
incidentreports	{ createdAt: 1 }	TTL (7d)	Auto-expiry

5. Security Architecture

5.1 Security Layers Overview

SecureOcte implements defense-in-depth with seven distinct security layers applied in middleware order:

Layer	Middleware	Mechanism	Failure Mode
1. Host Validation	Inline in server.js	Whitelist ALLOWED_HOST header	403 Forbidden
2. CORS Enforcement	cors()	Origin whitelist per route group	CORS preflight rejection
3. Security Headers	helmetConfig.js	CSP, HSTS, X-Frame-Options, referrer policy	N/A — always applied
4. Rate Limiting	distributedRateLimit.js	Redis-backed 120 req/min global; 60 tracking; 10 stream	503 if Redis down (fail-closed)
5. Authentication	verifyToken.js	JWT verification + strict device ID binding	401 Unauthorized
6. Replay Protection	replayProtection.js	X-Timestamp $\pm 30s$ + X-Nonce stored in Redis 60s	503 if Redis down (fail-closed)
7. GPS Anomaly Guard	gpsGuard.js / gpsAnomalyGuard.js	Speed check (mode-aware) + teleport detection	400 Bad Request

5.2 JWT Authentication

Algorithm: HMAC-SHA256 (jsonwebtoken default). Token expiry: 7 days.

Token Payload:

```
{ userId, role, username, deviceId, iat, exp }
```

Device Binding (verifyToken.js): For all non-admin users, the X-Device-Id header must exactly match the deviceId embedded in the JWT at login. Mismatch triggers a 401 and is logged as a forensic event. This prevents token theft across devices.

Admin Authentication: Admin tokens have role: 'admin' and are exempt from device binding. Admin credentials are validated against ADMIN_USERNAME and ADMIN_PASSWORD environment variables — no database user record is used.

5.3 Replay Protection

All sensitive mutation endpoints (panic trigger, location updates, cab trip updates, patrol updates) require two additional headers:

Header	Format	Validation
X-Timestamp	Unix milliseconds (numeric string)	Must be within $\pm 30,000$ ms of server time
X-Nonce	UUID or hex string, 8–128 chars	SET NX in Redis with 60s TTL; second use = 400 Duplicate

Redis failure behavior: FAIL CLOSED — returns 503 instead of allowing the request through.

5.4 Rate Limiting

Scope	Window	Max Requests	Key	Store
Global (all routes)	60 s	120	IP address	Redis (fail-closed)
Tracking (location updates)	60 s	60	userId or IP	Redis (fail-closed)
Stream endpoints	60 s	10	userId or IP	Redis (fail-closed)
Panic (window 1)	10 s	2	userId	Redis
Panic (window 2)	60 s	3	userId	Redis
Panic (window 3)	10 min	10	userId	Redis
Login	15 min	10	IP	In-memory
Signup	60 min	5	IP	In-memory
Admin Login	15 min	5	IP	In-memory
Incident Reporting	10 min	5	userId or IP	In-memory
Cab Trip Start	60 s	5	userId	In-memory

Rate-limit headers (X-RateLimit-*) are suppressed on all endpoints to prevent information disclosure.

5.5 GPS Anomaly Detection

Two middleware implementations provide GPS sanity checking. Both store last known position in Redis under `gps:{entityKey}` with 60-second TTL:

Check	Threshold	Action
Speed (mode-aware — <code>gpsGuard.js</code>)	walk: 20 km/h car: 160 km/h drone: 250 km/h default: 120 km/h	400 Bad Request
Speed (fixed — <code>gpsAnomalyGuard.js</code>)	> 200 km/h implied speed	400 Bad Request
Teleport	> 50 km jump AND elapsed < 30 s	400 Bad Request

Redis unavailability: both guards fail OPEN (let the request through) to avoid blocking genuine emergencies.

5.6 Security Headers (Helmet)

Header	Value	Purpose
Content-Security-Policy	self, maps.googleapis.com, maps.gstatic.com	XSS prevention
Strict-Transport-Security	max-age=31536000; includeSubDomains; preload	Force HTTPS
X-Frame-Options	DENY	Clickjacking prevention
Referrer-Policy	no-referrer	Referrer leakage prevention
X-Powered-By	Removed	Technology fingerprint suppression
x-render-origin-server	Removed	Infrastructure detail suppression
X-RateLimit-*	Removed	Rate limit state suppression

5.7 Anomaly Detection (anomalyDetection.js)

Detector	Threshold	Behavior
sosAbuseGuard	> 5 panics in 10 minutes	Blocks with 429; warn at 70% (3–4 panics)
unusualTimeGuard	01:00–04:00 UTC	Non-blocking; annotates req.anomaly.unusualTime = true; logged
locationJumpCounter	req.gpsAnomaly flag set	Increments Redis counter; logs to forensic stream

5.8 Audit Logging

All requests to /api/auth, /api/panic, /api/admin, and /health are captured by the audit logger (securityLogger.js) with the following fields: audit flag, HTTP method and path, status code, latency, IP address, userId, role, deviceId, and user-agent (truncated to 120 chars).

Admin mutation actions are additionally logged with adminAction descriptor and adminUser identity. Log entries are structured JSON via pino and should be shipped to an append-only sink with minimum 90-day retention.

6. Alert Processing Pipeline

6.1 Panic Trigger Flow

The panic processing pipeline is the core of SecureOcte. It processes SOS alerts asynchronously to ensure low-latency response to the user while guaranteeing reliable processing through retry logic.

15. Mobile app calls POST `/api/panic` with `{ latitude, longitude, alertReason, accuracy }` + security headers
16. `verifyToken` validates JWT and device binding
17. `replayProtection` validates X-Timestamp and X-Nonce against Redis
18. `panicLimiter` (3-window) enforces 2/10s, 3/1min, 10/10min per `userId`
19. `validate(panicSchema)` checks coordinate types and ranges via Zod
20. `panicGpsGuard` checks speed and teleport anomaly against cached last position
21. Handler generates UUID `alertId`, calls `triggerAlert()`
22. `triggerAlert()` checks queue capacity (max 500 waiting jobs); throws `QUEUE_UNAVAILABLE` if full
23. `enqueueAlert()` adds BullMQ job with `jobId=alertId` (deduplication), `priority=1`, `attempts=3`, exponential backoff 2s
24. Handler immediately responds 201 `{ message, alertId }` — user is not blocked waiting for DB write
25. Alert worker (concurrency 5) picks up the job: writes `UserLocation` + `Panic` to MongoDB simultaneously
26. On DB failure: job is re-queued with exponential backoff (2s → 4s → 8s); up to 3 attempts
27. Dashboard polls `/api/panic/locations` to display updated map

6.2 Queue Configuration

Parameter	Value	Rationale
Queue name	panic-alerts	Dedicated queue for isolation
Job ID	alertId (UUID v4)	Prevents duplicate processing of same alert
Priority	1 (highest)	Panic jobs preempt any other queue activity
Attempts	3	Balance between reliability and storm prevention
Backoff strategy	Exponential, 2s base	2s → 4s → 8s delays
Max queue depth	500 waiting + delayed jobs	503 returned if exceeded
Worker concurrency	5 jobs per process	Allows burst processing
<code>removeOnComplete</code>	100	Retain last 100 completed for debugging
<code>removeOnFail</code>	50	Retain last 50 failed for analysis

6.3 Live Location Tracking

After a panic is triggered, the mobile app begins sending live location updates at 5-second intervals via PUT `/api/panic/location`. The handler finds the existing `UserLocation` document by `{ alertId, userId }` and overwrites latitude/longitude in-place. The `Panic` document's location field is also updated to keep both collections consistent.

Rate limit on location updates: 60 requests/minute per `userId` (enforced by both `distributedRateLimit` `trackingLimiter` and a local `locationUpdateLimiter`).

7. Mobile Application

7.1 Architecture Overview

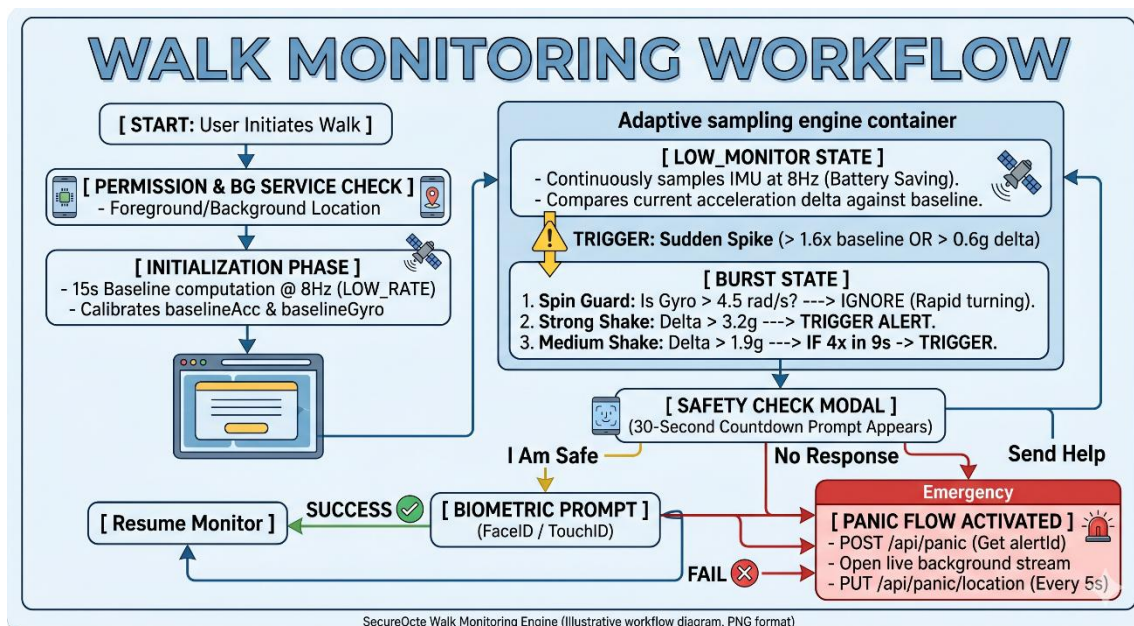
The mobile app is built with React Native (Expo SDK 54) and targets Android and iOS. It uses a custom hook (useWalkEngine) as the persistent safety engine that survives screen navigation, ensuring sensor subscriptions and GPS watchers never tear down mid-session.

7.2 Navigation Structure

Screen	File	Purpose
AuthScreen	AuthScreen.js	Login and signup forms; stores token in SecureStore
HomeScreen	HomeScreen.js	Main hub: SOS button, map with high-risk zones, voice trigger
WalkMonitoringScreen	WalkMonitoringScreen.js	Display layer for useWalkEngine; sensor readings, elapsed time
CabMonitoringScreen	CabMonitoringScreen.js	Cab trip tracking with route deviation detection
CabDestinationMapScreen	CabDestinationMapScreen.js	Google Places destination picker with map preview
AIChatScreen / ChatbotScreen	AIChatScreen.js	Gemini-powered Neha chatbot interface
VoiceCabAssistantScreen	VoiceCabAssistantScreen.js	Voice command → mode detection → auto-start trip
PatrolVehicleScreen	Patrolvehiclescreen.js	Officer patrol start, location update, dispatch acknowledgement
ReportIncidentScreen	Reportincidentscreen.js	Citizen incident reporting form
IncidentReportsScreen	IncidentReportsScreen.js	View submitted incident reports and their status
ProfileScreen	ProfileScreen.js	User profile management and trusted contacts
MapPickerScreen	MapPickerScreen.js	Generic map coordinate picker

7.3 Walk Monitoring Engine (useWalkEngine.js)

The walk engine is a React hook mounted at the App root level. It manages all sensors and never unmounts during navigation. Key characteristics:



Sensor Configuration

Sensor	Rate	Purpose
Accelerometer	80 ms (10 Hz) / 16 ms (60 Hz burst)	Impact and shake detection
Gyroscope	80 ms (10 Hz) / 16 ms (60 Hz burst)	Rotation detection
GPS (expo-location)	Background task every 5 s	Speed monitoring and location tracking

Adaptive Sampling

The engine uses two sampling states: LOW_MONITOR (8 Hz baseline) and BURST (60 Hz for 3 seconds). It transitions to BURST when sensor readings exceed $1.6\times$ the 15-second rolling baseline for accelerometer, or $1.5\times$ for gyroscope. This conserves battery during normal walking while ensuring high-resolution capture during suspicious events.

Detection Thresholds

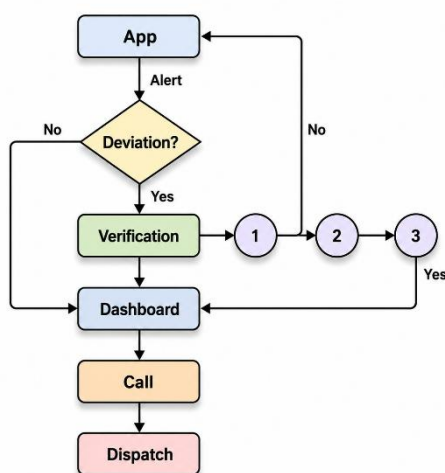
Signal	Threshold	Action
Strong shake	> 3.2 g delta acceleration	Immediate SOS trigger
Sustained medium shakes	> 1.9 g, 4 times in 9 s window	SOS trigger
Speed anomaly	> 15 km/h during walk session	Safety check notification
Max rotation	> 4.5 rad/s gyroscope magnitude	Contributing factor to SOS
Safety check no-response	User ignores check notification	SOS with reason SAFETY_CHECK_NO_RESPONSE
Safety check explicit no	User taps 'No, Send Help'	SOS with reason SAFETY_CHECK_NO

SOS Trigger Flow

28. Location preloaded asynchronously via `getReliableLocation()` (cascading accuracy fallback)
29. 15-second countdown modal displayed with animated ring progress
30. Background location fetch continues during countdown to ensure freshest coordinates
31. On countdown expiry (or user confirmation): POST `/api/panic` with cached location
32. `alertId` returned; 5-second interval begins sending PUT `/api/panic/location`
33. Interval clears when user explicitly stops monitoring or clears alert

7.4 Cab Monitoring

The cab escort feature monitors the user's journey against a declared destination. Key mechanisms:



- Destination set via Google Places Autocomplete (react-native-google-places-autocomplete)
- Route deviation detected by comparing current GPS coordinates against expected route corridor
- Three deviation notification actions: I'm Safe, Suspicious, Need Help
- Police Support button creates a NEW CabEscortTrip document with policeSupportRequested=true — ensures dashboard shows a separate alert card per trigger
- Location updates sent every 5 seconds via POST /api/cab-escort/update-location with GPS anomaly guard

7.5 AI Integration (Gemini 2.5 Flash)

Feature	Function	Behavior
Neha Chatbot	chatWithGemini(messages)	Safety companion persona; travel guidance; entertainment; no legal/medical advice; never says 'you are safe'
Voice Command Parser	parseTripCommand(text)	Returns { mode: CAB_MONITORING WALK_MONITORING UNKNOWN, destination: string null } as strict JSON
Mode Classifier	decideMonitoringMode(text)	Single-word response: WALK_MONITORING CAB_MONITORING SOS

7.6 Key Mobile Dependencies

Package	Version	Purpose
expo	~54.0.7	Managed workflow and native module access
expo-location	~19.0.8	GPS and background location tracking
expo-sensors	~15.0.7	Accelerometer and Gyroscope
expo-notifications	~0.32.16	Local and push notifications with action categories
expo-local-authentication	~17.0.8	Biometric authentication for SecureMe
expo-secure-store	~15.0.8	Encrypted token storage
expo-task-manager	~14.0.9	Background GPS task (walk-background-location)
expo-battery	~10.0.8	Battery level monitoring
react-native-maps	1.20.1	Map display with zone polygons and markers
socket.io-client	^4.8.3	WebSocket connection for real-time events
@react-native-community/netinfo	11.4.1	Network connectivity detection for offline handling
react-native-background-actions	^4.0.1	Background task execution on Android

8. API Reference

8.1 Authentication Endpoints

Method	Path	Auth	Request Body	Response
POST	/api/auth/signup	None	username, password, mobile, deviceId, gender?, dob?, trustedContacts?	201: { token, userId, username, mobile, gender, dob, age }
POST	/api/auth/login	None	mobile, password, deviceId	200: { token, userId, username, mobile, gender, dob, age }
POST	/api/auth/admin/login	None	username, password	200: { token }
GET	/api/auth/users	Admin JWT	—	200: [User]
PUT	/api/auth/update/:id	User JWT	username?, mobile?, trustedContacts?, gender?, dob?	200: Updated profile fields

8.2 Panic Endpoints

NOTE: All POST/PUT panic endpoints require X-Timestamp, X-Nonce, and X-Device-Id headers.

Method	Path	Auth	Request Body	Response
POST	/api/panic	User JWT	{ latitude, longitude, alertReason?, accuracy? }	201: { message, alertId }
PUT	/api/panic/location	User JWT	{ alertId, latitude, longitude }	200: { ok: true }
DELETE	/api/panic	User JWT	—	200: { message }
GET	/api/panic	Admin JWT	—	200: [Panic]
GET	/api/panic/locations	Admin JWT	—	200: [UserLocation]

8.3 Walk Session Endpoints

Method	Path	Auth	Request Body	Response
POST	/api/walk/start	User JWT	{ latitude, longitude, username? }	200: { session }
POST	/api/walk/stop/:id	User JWT	—	200: { session }
POST	/api/walk/restart/:id	User JWT	—	200: { session }
GET	/api/walk/active	User JWT	—	200: [WalkSession]

Method	Path	Auth	Request Body	Response
GET	/api/walk/stopped	User JWT	—	200: [WalkSession]
GET	/api/walk/history	User JWT	—	200: [WalkSession (normalized)]

8.4 Cab Escort Endpoints

Method	Path	Auth	Request Body / Param	Response
POST	/api/cab-escort/start	User JWT	{ destination: {latitude, longitude, address?}, vehicleNumber?, vehicleType?, currentLocation? }	200: { trip }
POST	/api/cab-escort/update-location	User JWT	{ tripId, latitude, longitude }	200: { ok }
POST	/api/cab-escort/police-support	User JWT	{ tripId }	200: { trip } (new alert document)
POST	/api/cab-escort/complete/:tripId	User JWT	—	200: { trip }
GET	/api/cab-escort/active	User JWT	—	200: [CabEscortTrip]
GET	/api/cab-escort/user/:userId	Owner or Admin JWT	—	200: [CabEscortTrip]
GET	/api/cab-escort/police-support	User JWT	—	200: [CabEscortTrip]
GET	/api/cab-escort/live/:tripId	User JWT	—	200: { currentLocation, updatedAt }
GET	/api/cab-escort/history	User JWT	—	200: [CabEscortTrip]

8.5 Patrol Endpoints

Method	Path	Auth	Notes
POST	/api/patrol/start	User JWT	Creates new PatrolTrip; officerId from JWT
POST	/api/patrol/update-location	User JWT + replayProtection + GPS guard	tripId + coordinates; 60 req/min limit
POST	/api/patrol/dispatch	Admin JWT	Sets dispatchedAlert on trip; sends Expo push notification

Method	Path	Auth	Notes
POST	/api/patrol/dispatch/acknowledge/:tripId	Officer JWT	Sets dispatchedAlert.acknowledged = true
POST	/api/patrol/complete/:tripId	Officer or Admin JWT	Sets status = completed
GET	/api/patrol/active	User JWT	All active patrol trips
GET	/api/patrol/officer/:officerId	Owner or Admin JWT	Officer's own trip history
GET	/api/patrol/live/:tripId	User JWT	Current location of a patrol vehicle
GET	/api/patrol/pending-dispatch/:tripId	User JWT	Polls for pending dispatch alert on this trip
GET	/api/patrol/history	User JWT	Completed patrol trips (last 500)

8.6 Incident Report Endpoints

Method	Path	Auth	Notes
POST	/api/incident/report	User JWT	Validates category, crowdSize, behaviorIndicators; finds nearest patrol by Haversine distance; sends push notification
GET	/api/incident/my-reports	User JWT	Last 50 reports by requesting user
GET	/api/incident/assigned/:tripId	Officer JWT	Pending + acknowledged reports assigned to this patrol trip
GET	/api/incident/all	Admin JWT	All incident reports (last 500)
GET	/api/incident/pending	Admin JWT	Pending reports only (last 200)
POST	/api/incident/acknowledge/:id	Officer or Admin JWT	Update status to acknowledged or resolved

8.7 Stream Endpoints

Method	Path	Auth	Notes
POST	/api/stream/create	Admin JWT + streamLimiter	Resolves userId to canonical form (panic session or cab trip); creates LiveStream document
GET	/api/stream/:streamId	Public + streamLimiter	Returns 404 if not found, 410 if expired, else { userId, latitude, longitude, alertId, updatedAt, source }

8.8 Statistics Endpoints

GET /api/stats/dashboard (Admin JWT only) — Returns aggregated counts for CabEscortTrip and WalkSession across:

- today, yesterday, week (rolling 7-day), lastWeek (prior 7-day), month (calendar month), lastMonth, lifetime

8.9 Health Endpoint

GET /health — Returns { status: ok|degraded, uptime, timestamp, checks: { db, redis, queue } }

Full check response requires request from 127.0.0.1 or Authorization: Bearer {HEALTH_TOKEN}. Unauthenticated callers receive only { status: ok } to prevent infrastructure disclosure.

8.10 Standard Error Responses

HTTP Status	Trigger	Response Body
400	Validation failure, bad timestamp, duplicate nonce, GPS anomaly	{ error: '<message>' }
401	Missing/invalid JWT, device mismatch, expired token	{ error: 'Unauthorized' }
403	Role insufficient (non-admin accessing admin route)	{ error: 'Unauthorized' 'Admin access only' }
404	Document not found	{ error: '...' }
410	Stream expired	{ error: 'Stream has expired' }
429	Rate limit exceeded	{ error: '<rate limit message>' }
500	Unhandled exception	{ error: 'Internal server error' }
503	Redis unavailable (fail-closed), queue at capacity, request timeout	{ error: '<reason>' }

9. AI Module (Gemini Integration)

9.1 Overview

The AI module (`gemini.js`) integrates Google Gemini 2.5 Flash for three distinct functions within the SecureOcte platform. All Gemini calls are made server-side using the `@google/genai` SDK.

9.2 Neha Chatbot

Neha is a safety-focused AI companion persona. She accompanies users during travel, providing emotional support, safety guidance, and entertainment. Persona rules enforced via system prompt:

- Identity: 'I am Neha, an AI created by SecureOcte' — never admits to being Gemini or Google
- Never says 'you are safe' (avoids false reassurance)
- Never provides legal or medical advice
- Never instructs user to confront a threat
- Maintains calm, empathetic, non-judgmental tone
- Actively entertains users during anxious travel: stories, jokes, movie discussions

9.3 Voice Command Parser (`parseTripCommand`)

Parses free-form voice input into structured trip intent. Returns strict JSON only:

```
{ "mode": "CAB_MONITORING | WALK_MONITORING | UNKNOWN", "destination": "<place name or null>" }
```

Classification rules: `cab/auto/taxi/ride/trip/drop me` → `CAB_MONITORING`; `walk/walking/on foot` → `WALK_MONITORING`. Defensive JSON extraction via `indexOf('{')` / `lastIndexOf('')` to handle Gemini preamble.

9.4 Mode Classifier (`decideMonitoringMode`)

Single-word classifier for voice commands. Returns exactly one of: `WALK_MONITORING` | `CAB_MONITORING` | `SOS`. `SOS` is triggered by: `help`, `emergency`, `danger`, `call police`, `save me`, `I'm scared`, and similar distress phrases.

10. Operator Dashboard

10.1 Overview

The operator dashboard (Dashboard/Dashboard.html) is a lightweight static HTML + JavaScript interface. It communicates directly with backend REST APIs and requires no build process. Designed for low overhead and fast loading in control room environments.

10.2 Feature Set

Feature	Implementation	API Endpoint
Live alert monitoring	Polling interval display of active panics and locations	GET /api/panic/locations
Alert lifecycle management	Status transition: New → Acknowledged → Resolved	Status updates via patrol and incident routes
Priority pinning	Highlight critical alerts with pin toggle	Client-side state
Track user	Open Google Maps with user coordinates	Data from /api/panic/locations
Call user	tel: link to user mobile number	Mobile from user profile
Start live stream	Creates stream and shows viewer URL	POST /api/stream/create
Forward to authorities	Admin dispatch to nearest patrol vehicle	POST /api/patrol/dispatch
Map visualization	Google Maps API with real-time markers	Data from location endpoints
Statistics dashboard	Today/week/month counters for cab and walk activity	GET /api/stats/dashboard
Incident management	View, assign, and resolve citizen-reported incidents	GET /api/incident/all

11. Deployment & Operations

11.1 Backend Deployment

PM2 Cluster Configuration (ecosystem.config.cjs)

Parameter	Value	Purpose
instances	max	One worker per CPU core
exec_mode	cluster	Shared port, built-in load balancing
autorestart	true	Restart on uncaught exception
max_restarts	10	Circuit breaker to prevent restart storms
restart_delay	2000 ms	Prevents rapid restart loops
max_memory_restart	512 MB	Memory leak protection
kill_timeout	5000 ms	Graceful shutdown window
listen_timeout	8000 ms	Startup health check timeout
log_date_format	YYYY-MM-DD HH:mm:ss Z	Timestamped log entries

Startup Commands

```
pm2 start ecosystem.config.cjs
pm2 reload ecosystem.config.cjs --update-env      # Zero-downtime reload
pm2 logs safety-backend                            # Tail live logs
```

11.2 Mobile App Deployment

Step	Command	Notes
Install dependencies	cd mobile-app && npm install	Uses yarn.lock for reproducibility
Start development server	npx expo start	Metro bundler; scan QR with Expo Go
Build Android	npx expo run:android	Requires Android SDK
Build iOS	npx expo run:ios	Requires macOS + Xcode
EAS production build	eas build	Uses eas.json config for managed builds

11.3 Health Monitoring

The /health endpoint provides structured dependency checks for DB, Redis, and queue:

```
{ "status": "ok|degraded", "uptime": 3600, "timestamp": "...", "checks": { "db": {
"status": "ok", "latencyMs": 4 }, "redis": { "status": "ok" }, "queue": { "status":
"ok", "counts": { "waiting": 0, "active": 2, "failed": 0 } } } }
```

HTTP 200 = all healthy; HTTP 503 = one or more dependencies degraded. Integrate with your uptime monitoring tool (e.g., UptimeRobot, Datadog Synthetics) to alert on 503.

11.4 Redis Operational Notes

- Single-node mode: set REDIS_URL=redis://:password@host:6379
- Cluster mode: set REDIS_CLUSTER=true and REDIS_CLUSTER_NODES=host1:6379,host2:6379,host3:6379
- TLS: set REDIS_TLS=true for Redis clusters requiring encrypted connections
- Redis health is checked every 30 seconds via passive ping; reconnect uses exponential backoff capped at 10 seconds, max 10 attempts
- Key namespace conventions: replay:{nonce}, gps:{entityKey}, rl:{scope}:{key}, anomaly:{type}:{userId}

11.5 Log Retention Requirements

Per the security design (securityLogger.js), audit logs must be retained for a minimum of 90 days. Configure your log shipping pipeline accordingly:

- PM2 log files: ./logs/out.log and ./logs/error.log
- Recommended: ship JSON logs to CloudWatch Logs, Datadog, Loki, or equivalent with 90-day retention policy
- Apply log rotation to prevent disk exhaustion (pm2-logrotate module recommended)

12. Known Issues & Future Roadmap

12.1 Known Issues

ID	Severity	Description	Affected File	Status
KI-001	Low	Dead code in gemini.js: parseTripCommand() calls logger after return statement — unreachable code	backend/gemini.js	Open
KI-002	Low	GEMINI_API_KEY is hardcoded as placeholder string in gemini.js; must be moved to environment variable before production deployment	backend/gemini.js	Open
KI-003	Low	HighRiskZone model uses CommonJS require() while all other models use ES Modules — inconsistency	backend/models/HighRiskZone.js	Open
KI-004	Medium	No dead-letter queue (DLQ) for BullMQ: jobs that exhaust all 3 retry attempts are silently discarded after removeOnFail retention	backend/queues/alertQueue.js	Open
KI-005	Low	Admin login uses plaintext credential comparison against env vars; should use bcrypt-hashed stored credentials for production hardening	backend/routes/auth.js	Open
KI-006	Info	streamLimiter in distributedRateLimit.js exports an array; stream.js imports the single middleware streamLimiter.js — naming collision risk	backend/middlewares/	Monitored

12.2 Planned Roadmap

Priority	Feature	Description
High	Dead-letter queue	BullMQ DLQ for failed jobs — enables manual requeue and forensic analysis of dropped alerts
High	Refresh token rotation	Full refresh token flow using the existing refreshTokens schema field — currently schema is defined but endpoint is not implemented
Medium	Geo-based nearest patrol dispatch	Replace Haversine-only nearest-patrol selection with road-distance or travel-time routing via Google Maps Distance Matrix API

Priority	Feature	Description
Medium	Multi-region deployment	Geographic failover with Redis replication across regions
Medium	WebSocket real-time push	Replace dashboard polling with Socket.IO event push for sub-second alert display
Low	AI anomaly detection	ML-based behavioral analysis to supplement rule-based anomaly detection
Low	Admin credential hardening	bcrypt-hash admin passwords stored in DB rather than plaintext env vars
Low	Expo push notification retry	Handle Expo push delivery failures with receipt checking and retry queue

13. Setup & Installation Guide

13.1 Prerequisites

Requirement	Version	Notes
Node.js	>= 18.x LTS	Required for ESM support (type: module in package.json)
npm	>= 9.x	Comes with Node.js
MongoDB Atlas	Any cluster tier	Free tier works for development
Redis	>= 6.x	Local or managed (Redis Cloud, Upstash)
PM2	Latest	npm install -g pm2 for production deployment
Expo CLI	Latest	npx expo — no global install required
Android Studio / Xcode	Latest	Only required for native builds

13.2 Backend Setup

34. Clone the repository and navigate to the backend directory

```
git clone https://github.com/your-username/SecureOcte.git
cd SecureOcte/backend
```

35. Install dependencies

```
npm install
```

36. Create .env file with all required environment variables (see Section 2.3)

37. Start in development mode

```
npm run dev # Uses nodemon for hot-reload
```

38. Start in production mode

```
pm2 start ecosystem.config.cjs
```

13.3 Mobile App Setup

39. Navigate to the mobile-app directory

```
cd mobile-app
```

40. Install dependencies

```
npm install
```

41. Update the BACKEND_URL constant in screens/useWalkEngine.js and screens/api.js to point to your deployed backend

42. Start Expo development server

```
npx expo start
```

43. For production builds, configure app.json and eas.json, then run eas build

13.4 Validation Checklist

Before going live, verify the following:

- GEMINI_API_KEY moved from hardcoded string to GEMINI_API_KEY environment variable in gemini.js
- HEALTH_TOKEN set in environment to protect /health diagnostic output
- ALLOWED_HOST set to your production domain
- CORS_ORIGIN and EXTRA_CORS_ORIGINS configured for your frontend origins
- Redis accessible and REDIS_URL correctly set; verify /health returns redis.status: ok
- MongoDB Atlas IP whitelist includes your server's egress IPs
- PM2 log rotation configured (pm2 install pm2-logrotate)
- Log retention policy set to minimum 90 days in your log aggregator
- Expo push notification receipts checked after first live dispatch

14. Glossary

Term	Definition
BullMQ	Redis-backed job queue library for Node.js supporting priorities, retries, and deduplication
CSP	Content Security Policy — HTTP header controlling which resources a browser can load
DLQ	Dead-Letter Queue — secondary queue for jobs that have exhausted all retry attempts
Expo Push Token	Device-specific token (ExponentPushToken[...]) used to send push notifications via Expo's push service
Fail Closed	Security posture where a component blocks all requests when a dependency (e.g., Redis) is unavailable
Fail Open	Security posture where a component allows requests when a dependency is unavailable (used for GPS guards to avoid blocking emergencies)
Haversine Formula	Mathematical formula to calculate great-circle distance between two GPS coordinates on Earth's surface
HSTS	HTTP Strict Transport Security — instructs browsers to use HTTPS only for a domain
Idempotency	Property of an operation where applying it multiple times produces the same result (enforced via alertId as BullMQ jobId)
JWT	JSON Web Token — compact, self-contained token for transmitting claims between parties
PM2	Process Manager 2 — Node.js production process manager with cluster mode and log management
Replay Attack	An attack where a valid captured request is re-sent to trigger the same action again
SecureMe	Feature that periodically prompts biometric verification; absence of response triggers an SOS
Sensor Fusion	Combining data from multiple sensors (GPS, accelerometer, gyroscope) to make more accurate inferences
SOS	Save Our Souls — distress signal; in SecureOcte, refers to a panic/emergency alert
TTL Index	MongoDB Time-To-Live index that automatically deletes documents after a specified duration
WebSocket	Full-duplex communication protocol over a single TCP connection; used for real-time dashboard updates